모바일&스마트시스템

기말프로젝트_최종보고서

2024-12-12

담당 교수: 송미화 교수님

분반 : N

학번 : 2171456

이름 : 임재윤

라즈베리파이를 이용한 스쿼트 자세 교정 시스템 결과 보고서

2171456 임재윤

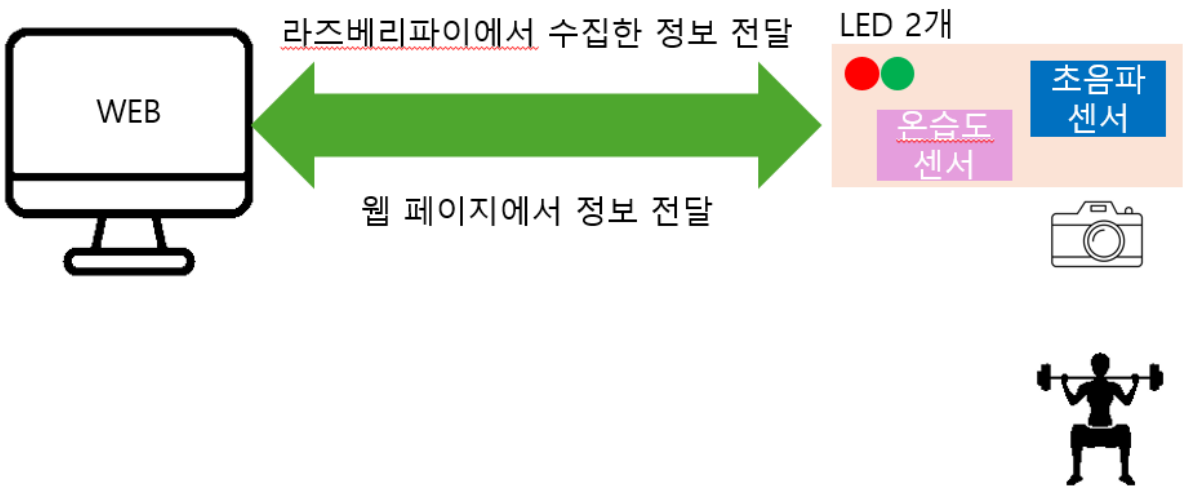
1. 작품 개요

스쿼트를 올바른 자세로 할 수 있게 만들어주는 시스템이다. 라즈베리파이에 카메라, 초음파 센서, 온습도 센서, LED를 연결하여 웹 페이지를 통해 집, 헬스장 등에서 자신의 스쿼트 자세를 실시간으로 확인할 수 있다. 올바른 자세로 해야 카운트가 되고, 자세가 잘못되어 있는 경우에는 자세에 맞는 피드백을 제공한다.

첫 번째로 초음파 센서를 통해 스쿼트 반복 횟수 추적이 가능하다. 카메라를 통해 상체 기울기, 무릎 위치 등을 실시간으로 분석하면서 자세가 잘못될 경우 초음파 센서가 이를 감지할 수 있게 된다. 만약 올바른 자세로 인식하여 카운트를 성공적으로 했을 경우, LED의 색깔이 빨강색에서 초록색으로 잠시 변한다.

두 번째로 온습도 센서를 통해 운동 전후의 환경 변화를 확인할 수 있다. 스쿼트를 한 후 온도와 습도의 변화를 기록하여 운동 강도와 체력 소모 추정이 가능하다.

<시스템 구조>



2. 구현 방법

2.1 하드웨어 부분

라즈베리파이에 카메라 1대, 초음파 센서 1개, 온습도 센서 1개, LED 2개를 사용한다.

GPIO 핀은 다음과 같다.

LED - GPIO5, GPIO6

초음파 센서 - GPIO16, GPIO20

온습도 센서 – SDA1, SCL1

2.2 소프트웨어 부분

2.2.1 스쿼트 동작 실시간 감지, 비디오 스트리밍, 온습도 센서, 초음파 센서, LED를 제어하는 파이썬 코드 (squat.py)

- Flask를 사용하여 웹 서버를 구축하고, MediaPipe를 이용해 스쿼트 동작을 분석하며, Raspberry Pi GPIO를 통해 LED를 제어한다.

```
import os
import pygame
import time
import cv2
import numpy as np
from datetime import datetime
import threading
from flask import Flask, render_template, Response, send_from_directory, jsonify
from flask_socketio import SocketIO, emit
from flask_cors import CORS
import mediapipe as mp
import RPi.GPIO as GPIO
import board
```

```
import busio
import smbus2
from adafruit_htu21d import HTU21D

# Flask 애플리케이션 설정
app = Flask(__name__)
CORS(app)
socketio = SocketIO(app)
HTU21D_ADDR = 0x40
TEMP_MEASURE_CMD = 0xE3
HUMIDITY_MEASURE_CMD = 0xE5
bus = smbus2.SMBus(1)

# 전역 변수
show_timer = False
in_squat = False
squat_count = 0
stop_streaming = False  # 스트리밍 종료 플래그 추가
current_video_filename = None  # 비디오 파일 이름 전역 변수 추가
VIDEO_DIRECTORY = os.path.join(os.getcwd(), 'static', 'squat_videos')  # 서버 내 경로로 수정
count_dir = 'static/squat_count'

if not os.path.exists(VIDEO_DIRECTORY):
    os.makedirs(VIDEO_DIRECTORY)
```

```
if not os.path.exists(count_dir):
    os.makedirs(count_dir)

# MediaPipe 설정
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
mp_drawing = mp.solutions.drawing_utils

# pygame 초기화 (소리 재생을 위한)
pygame.mixer.init()

# GPIO 설정
GPIO.setmode(GPIO.BCM)
GPIO.setup(5, GPIO.OUT)  # GPIO5 LED
GPIO.setup(6, GPIO.OUT)  # GPIO6 LED

# 스쿼트 동작 분석 함수
def start_squat_detection():
    global squat_count, current_video_filename, in_squat, stop_streaming, squat_goal
    if squat_goal is None:
        squat_goal = 5
    current_time = datetime.now().strftime("%m%d_%H%M")
    current_video_filename = f"squat_{current_time}.avi"  # .avi 확장자로 변경
    filename = os.path.join(VIDEO_DIRECTORY, current_video_filename)
```

```
count_file = os.path.join(count_dir, f"squat_{current_time}.txt")
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
height, width, _ = frame.shape

# XVID 코덱을 사용한 비디오 압축 설정
fourcc = cv2.VideoWriter_fourcc(*'XVID')  # XVID 코덱 사용
out = cv2.VideoWriter(filename, fourcc, 8.0, (width, height))

if os.path.exists(count_file):
    with open(count_file, "r") as file:
        squat_count = int(file.read())
else:
    squat_count = 0

def play_beep():
    beep_sound = pygame.mixer.Sound('beep.wav')  # beep.wav 파일 사용
    beep_sound.play()

def calculate_angle(a, b, c):
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)
    radian = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a[0] - b[0])
    angle = np.abs(radian * 180.0 / np.pi)
```

```
    if angle > 180.0:
        angle = 360 - angle
    return angle

def get_body_angles(landmarks):
    knee = landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value]
    hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]
    ankle = landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value]
    knee_coord = (knee.x, knee.y)
    hip_coord = (hip.x, hip.y)
    ankle_coord = (ankle.x, ankle.y)
    knee_angle = calculate_angle(ankle_coord, knee_coord, hip_coord)

    shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value]
    torso = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]
    shoulder_coord = (shoulder.x, shoulder.y)
    torso_coord = (torso.x, torso.y)
    vertical_point = (torso.x, torso.y - 0.1)
    body_angle = calculate_angle(shoulder_coord, torso_coord, vertical_point)

    return knee_angle, body_angle

GPIO.output(5, GPIO.HIGH)  # 스트리밍 동안 GPIO5 LED 켜기
```

```
try:
    while cap.isOpened() and not stop_streaming:
        ret, frame = cap.read()
        if not ret:
            break

        image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(image_rgb)

        if results.pose_landmarks:
            landmarks = results.pose_landmarks.landmark
            knee_angle, body_angle = get_body_angles(landmarks)

            # 스쿼트 동작 감지
            if knee_angle >= 150 and not in_squat:
                in_squat = True
            elif in_squat:
                if knee_angle <= 95:
                    squat_count += 1
                    beep_thread = threading.Thread(target=play_beep)
                    beep_thread.start()
                    in_squat = False

                    # 스쿼트 성공 인식 시 GPIO 제어
                    GPIO.output(5, GPIO.LOW)  # GPIO5 LED 끄기
```

```
                    GPIO.output(6, GPIO.HIGH)  # GPIO6 LED 켜기
                    time.sleep(1)  # 1초 동안 유지
                    GPIO.output(6, GPIO.LOW)  # GPIO6 LED 끄기
                    GPIO.output(5, GPIO.HIGH)  # GPIO5 LED 켜기

                # 관절 표시
                if knee_angle <= 95:
                    mp_drawing.draw_landmarks(frame, results.pose_landmarks,
mp_pose.POSE_CONNECTIONS, mp_drawing.DrawingSpec(color=(0, 225, 0), thickness=2,
circle_radius=2), mp_drawing.DrawingSpec(color=(0, 225, 0), thickness=2, circle_radius=2))
                else:
                    mp_drawing.draw_landmarks(frame, results.pose_landmarks,
mp_pose.POSE_CONNECTIONS, mp_drawing.DrawingSpec(color=(0, 0, 255), thickness=2,
circle_radius=2), mp_drawing.DrawingSpec(color=(0, 0, 255), thickness=2, circle_radius=2))

                frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)
                frame = cv2.flip(frame, 1)

                # 타이머 및 피드백 표시
                frame = draw_text(frame, f'Knee Angle: {int(knee_angle)}', (10, 50))
                frame = draw_text(frame, f'Body Angle: {int(body_angle)}', (10, 100))
                frame = draw_text(frame, f'Squat Count: {squat_count}', (10, 150))

            # 여기서 프레임을 그대로 비디오로 저장
            out.write(frame)
            ref, buffer = cv2.imencode('.jpg', frame)
```

```
            frame = buffer.tobytes()
            yield (b'--frame₩r₩n'
                   b'Content-Type: image/jpeg₩r₩n₩r₩n' + frame + b'₩r₩n')

            # 스쿼트 설정 횟수 달성 시 스트리밍 종료 및 리디렉션
            if squat_count >= squat_goal:  # 스쿼트 설정 횟수 완료
                print("스쿼트 {squat_goal}회 달성, 종료합니다.")
                socketio.emit('redirect', {'redirect': True})
                GPIO.output(5, GPIO.LOW)
                break

    finally:
        # 비디오 파일이 제대로 닫히도록 하기 위해 잠시 대기
        out.release()  # 비디오 녹화 종료
        cap.release()  # 비디오 캡처 종료
        cv2.destroyAllWindows()

        # 프로그램 종료 시 카운트를 파일에 저장
        with open(count_file, "w") as file:
            file.write(str(squat_count))



def draw_text(frame, text, position):
    font = cv2.FONT_HERSHEY_SIMPLEX
```

```
    cv2.putText(frame, text, position, font, 1, (255, 0, 0), 2, cv2.LINE_AA)
    return frame

def read_temperature():
    bus.write_byte(HTU21D_ADDR, TEMP_MEASURE_CMD)
    time.sleep(0.5)  # 측정 대기 시간
    raw_temp = bus.read_word_data(HTU21D_ADDR, 0x00)  # 온도 데이터 읽기
    raw_temp = ((raw_temp & 0xFF) << 8) | (raw_temp >> 8)  # 바이트 순서 변경
    temperature = -46.85 + (175.72 * raw_temp) / 65536.0  # 온도 계산
    return round(temperature, 2)

def read_humidity():
    bus.write_byte(HTU21D_ADDR, HUMIDITY_MEASURE_CMD)
    time.sleep(0.1)  # 측정 대기 시간
    raw_humidity = bus.read_word_data(HTU21D_ADDR, 0x00)  # 습도 데이터 읽기
    raw_humidity = ((raw_humidity & 0xFF) << 8) | (raw_humidity >> 8)  # 바이트 순서
변경
    humidity = -6 + (125.0 * raw_humidity) / 65536.0  # 습도 계산
    return round(humidity, 2)

TRIG_PIN = 20
ECHO_PIN = 16

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG_PIN, GPIO.OUT)
```

```
GPIO.setup(ECHO_PIN, GPIO.IN)

def get_distance():
    GPIO.output(TRIG_PIN, GPIO.LOW)
    time.sleep(0.1)
    GPIO.output(TRIG_PIN, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG_PIN, GPIO.LOW)

    pulse_start = time.time()
    pulse_end = time.time()

    while GPIO.input(ECHO_PIN) == 0:
        pulse_start = time.time()

    while GPIO.input(ECHO_PIN) == 1:
        pulse_end = time.time()

    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150  # 초음파 속도 (초당 343 m)

    return round(distance, 2)

@app.route('/sensor-data', strict_slashes=False)
def get_sensor_data():
```

```
    temperature = read_temperature()
    humidity = read_humidity()
    distance = get_distance()
    data = {
        'temperature': temperature,
        'humidity': humidity,
        'distance': distance
    }
    return jsonify(data)

# Flask 라우트 설정
@app.route('/', strict_slashes=False)
def index():
    return render_template('index.html')

@app.route('/bye_squat', strict_slashes=False)
def bye_squat():
    return render_template('bye_squat.html')

@app.route('/serve_video/<filename>', strict_slashes=False)
def serve_video(filename):
    video_path = os.path.join('static', 'squat_videos', filename)  # 로컬 경로로 수정

    if os.path.exists(video_path):
        return send_from_directory('static/squat_videos', filename)
```

```
    else:
        return "File not found", 404

@app.route('/feedback_squat', strict_slashes=False)
def feedback_squat():
    global current_video_filename  # 전역 변수 사용
    if current_video_filename is None:
        return "Error: No video recorded", 404
    return render_template('feedback_squat.html', video_filename=current_video_filename)

@app.route('/squat', strict_slashes=False)
def squat():
    return render_template('squat.html')

@app.route('/timer_squat', strict_slashes=False)
def timer_squat():
    return render_template('timer_squat.html')

@app.route('/stream')
def stream():
    return Response(start_squat_detection(), mimetype='multipart/x-mixed-replace; boundary=frame')
# 전역 변수로 목표 스쿼트 횟수 설정
squat_goal = None
```

```
@app.route('/set_squat_goal', methods=['POST'])
def set_squat_goal():
    global squat_goal
    squat_goal = int(request.form['goal'])  # 목표 횟수를 전역 변수로 설정
    print(f"목표 스쿼트 횟수: {squat_goal}")
    return redirect(url_for('feedback_squat'))  # 피드백 페이지로 리디렉션

# 서버 실행
if __name__ == '__main__':
    sensor_thread = threading.Thread(target=get_sensor_data)
    sensor_thread.daemon = True
    sensor_thread.start()
    socketio.run(app, host='0.0.0.0', port=5000, debug=True)
```

2.2.2 스쿼트 첫 화면, 시작 웹페이지 (index.html)

- 스마트 헬스케어 웹사이트의 기본 레이아웃을 구성하는 페이지이며, '스쿼트 시작!' 버튼을 누르면 카운트다운 후 스쿼트 교정 화면으로 넘어간다.

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HELLNADO - 당신의 스마트 헬스케어 파트너</title>
    <link
```

```
    rel="stylesheet"
    href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css"
/>
<style>
    :root {
        --primary-color: #007aff;
        --background-color: #000;
        --text-color: #f5f5f7;
    }
    body,
    html {
        margin: 0;
        padding: 0;
        font-family: "SF Pro KR", "SF Pro Text", "SF Pro Icons", "Apple Gothic",
            "HY Gulim", "MalgunGothic", "HY Dotum", "Lexi Gulim", "Helvetica Neue",
            "Helvetica", "Arial", sans-serif;
        background-color: var(--background-color);
        color: var(--text-color);
        line-height: 1.6;
    }
    .container {
        max-width: 1200px;
        margin: 0 auto;
        padding: 0 20px;
    }
```

```
.nav-wrapper {
    background-color: rgba(0, 0, 0, 0.8);
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    z-index: 9999;
}
nav {
    display: flex;
    justify-content: flex-start;
    align-items: center;
    height: 70px;
    padding: 0 20px;
}

nav .logo {
    font-size: 28px;
    font-weight: bold;
    display: flex;
    align-items: center;
}

nav .menu {
    display: flex;
```

```
    align-items: center;
    margin-left: auto;
}

nav a {
    color: var(--text-color);
    text-decoration: none;
    font-size: 18px;
    opacity: 0.8;
    transition: opacity 0.3s ease, transform 0.3s ease;
    display: flex;
    align-items: center;
}
nav a:hover {
    opacity: 1;
    transform: scale(1.05);
}
nav a i {
    margin-right: 5px;
}
main {
    padding-top: 70px;
}
.hero {
    position: relative;
```

```
            height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            text-align: center;
            color: #fff;
            background-image: url("https://images.unsplash.com/photo-1517836357463-d25dfeac3438?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=1740&q=80");
            background-size: cover;
            background-position: center;
            overflow: hidden;
        }
        .hero::before {
            content: "";
            position: absolute;
            top: 0;
            left: 0;
            right: 0;
            bottom: 0;
            background: rgba(0, 0, 0, 0.5);
        }
        .hero-content {
            position: relative;
            z-index: 2;
```

```
    max-width: 800px;
    padding: 20px;
}
h1 {
    font-size: 64px;
    line-height: 1.07143;
    font-weight: 600;
    letter-spacing: -0.005em;
    margin-bottom: 20px;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
}
h2 {
    font-size: 32px;
    line-height: 1.10722;
    font-weight: 400;
    letter-spacing: 0.004em;
    text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.5);
    margin-bottom: 30px;
}
h3 {
    font-size: 50px;
    line-height: 1.10722;
    font-weight: 400;
    letter-spacing: 0.004em;
    text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.5);
```

```
    margin-bottom: 30px;
}
.smart-healthcare {
    padding: 100px 0;
    background-color: #1a1a1a;
}
.smart-healthcare-content {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
}
.smart-healthcare-text {
    flex: 1;
    min-width: 300px;
    padding-right: 40px;
}
.smart-healthcare-image {
    flex: 1;
    min-width: 300px;
    text-align: center;
}
.smart-healthcare-image img {
    max-width: 100%;
    height: auto;
```

```
    border-radius: 10px;
}
.features {
    padding: 100px 0;
    background-color: var(--background-color);
}
.features-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 40px;
}
.feature {
    text-align: center;
    padding: 20px;
    background-color: rgba(255, 255, 255, 0.05);
    border-radius: 10px;
    transition: transform 0.3s ease;
}
.feature:hover {
    transform: translateY(-10px);
}
.feature img {
    width: 150px;
    height: 150px;
    object-fit: cover;
```

```
    border-radius: 50%;
    margin-bottom: 20px;
}
.animate-on-scroll {
    opacity: 0;
    transform: translateY(20px);
    transition: opacity 0.8s ease, transform 0.8s ease;
}
.animate-on-scroll.is-visible {
    opacity: 1;
    transform: translateY(0);
}
@media (max-width: 768px) {
    h1 {
        font-size: 48px;
    }
    h2 {
        font-size: 24px;
    }
    .smart-healthcare-text,
    .smart-healthcare-image {
        flex: 100%;
        padding-right: 0;
        margin-bottom: 40px;
    }
```

```
    }
  </style>
</head>
<body>
  <div class="nav-wrapper">
    <nav>
      <a href="#" onclick="window.open('timer_squat/')"
        ><i class="fas fa-dumbbell"></i> 스쿼트 시작!</a
      >
      <div class="menu">
        <a href="#" onclick="window.open('feedback_squat/')"
          ><i class="fas fa-play"></i> </a
        >
      </div>
    </nav>
  </div>
  <main>
    <section class="hero">
      <div class="hero-content animate-on-scroll">
        <h1>HELLNADO</h1>
        <h2>당신의 스마트 헬스케어 파트너</h2>
        <h3>발판위에 서서 운동을 시작해주세요</h3>
      </div>
    </section>
    <section class="smart-healthcare">
```

```
<div class="container">
  <div class="smart-healthcare-content animate-on-scroll">
    <div class="smart-healthcare-text">
      <h2>스마트 헬스케어의 필요성과 의미</h2>
      <p>
        스마트 헬스케어는 현대 사회에서 더욱 중요해지고 있습니다. 바쁜
        일상 속에서 건강 관리의 어려움, 개인별 맞춤 건강 솔루션의
        필요성, 그리고 예방 중심의 건강 관리 트렌드가 증가하면서 스마트
        헬스케어의 역할이 커지고 있습니다.
      </p>
      <p>
        HELLNADO는 최신 기술을 활용하여 개인의 건강 상태를 실시간으로
        모니터링하고, 데이터를 분석하여 맞춤형 건강 관리 서비스를
        제공합니다. 이를 통해 더 효과적이고 개인화된 건강 관리가
        가능해집니다.
      </p>
    </div>
    <div class="smart-healthcare-image">
      <img
        src="https://images.unsplash.com/photo-1580086319619-
3ed498161c77?ixlib=rb-
4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=for
mat&fit=crop&w=1740&q=80"
        alt="스마트 헬스케어"
      />
    </div>
```

```
        </div>
      </div>
    </section>
    <section class="features">
      <div class="container">
        <h2 class="animate-on-scroll">HELLNADO의 주요 기능</h2>
        <div class="features-grid">
          <div class="feature animate-on-scroll">
            <img
              src="https://images.unsplash.com/photo-1541534741688-6078c6bfb5c5?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=1169&q=80"
              alt="개인 맞춤 운동"
            />
            <h3>개인 맞춤 운동</h3>
            <p>
              당신의 몸 상태와 목표에 맞는 최적의 운동 프로그램을 제공합니다.
            </p>
          </div>
          <div class="feature animate-on-scroll">
            <img
              src="https://images.unsplash.com/photo-1554244933-d876deb6b2ff?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=880&q=80"
              alt="실시간 자세 교정"
```

```
                />
                <h3>실시간 자세 교정</h3>
                <p>
                  AI 기술을 활용한 실시간 자세 교정으로 효과적이고 안전한 운동을
                  도와드립니다.
                </p>
              </div>
              <div class="feature animate-on-scroll">
                <img
                  src="https://images.unsplash.com/photo-1526506118085-
60ce8714f8c5?ixlib=rb-
4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=for
mat&fit=crop&w=687&q=80"
                  alt="전문가 코칭"
                />
                <h3>전문가 코칭</h3>
                <p>
                  경험 많은 전문 코치들의 1:1 온라인 코칭으로 더 나은 결과를
                  얻으세요.
                </p>
              </div>
            </div>
          </div>
        </section>
      </main>
      <script>
```

```
const animateOnScrollObserver = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add("is-visible");

        setTimeout(() => {
          entry.target.classList.remove("is-visible");
          requestAnimationFrame(() => {
            entry.target.classList.add("is-visible");
          });
        }, 50);
      } else {
        entry.target.classList.remove("is-visible");
      }
    });
  },
  {
    threshold: 0.1,
    rootMargin: "0px 0px -10% 0px",
  }
);

document.querySelectorAll(".animate-on-scroll").forEach((element) => {
  animateOnScrollObserver.observe(element);
```

```
        });

        // 페이지가 로드될 때 플래그 확인
        function checkRedirect() {
            fetch("/check_redirect")
                .then((response) => response.json())
                .then((data) => {
                    if (data.redirect) {
                        window.location.href = "/timer/";
                    }
                });
        }

        // 페이지 로드 시 플래그 확인
        document.addEventListener("DOMContentLoaded", checkRedirect);
    </script>
</body>
</html>
```

2.2.3 스쿼트 카운트 다운 웹페이지 (timer_squat.html)

- 스쿼트를 시작하기 전 카운트다운(3초 타이머)을 해주며, 카운트다운이 끝나면 스쿼트 교정 화면으로 넘어간다.

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>활기찬 3초 타이머</title>
  <style>
    body {
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
      background-color: #1a1a2e;
      font-family: Arial, sans-serif;
      overflow: hidden;
    }
    .timer-container {
      position: relative;
      width: 350px;
      height: 350px;
    }
    .rotating-circle {
      position: absolute;
      width: 100%;
      height: 100%;
      border: 4px solid #16213e;
```

```
    border-top: 4px solid #4ade80;
    border-radius: 50%;
    animation: rotate 3s linear infinite;
}
@keyframes rotate {
    0% {
        transform: rotate(0deg);
    }
    100% {
        transform: rotate(360deg);
    }
}
@keyframes fadeOutSlow {
    0% {
        opacity: 1;
    }
    100% {
        opacity: 0;
    }
}
#timer {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
```

```
            font-size: 8rem;
            font-weight: bold;
            color: #4ade80;
            text-shadow: 0 0 10px rgba(74, 222, 128, 0.5);
            text-align: center;
            white-space: nowrap;
        }
        .pulse {
            animation: pulse 0.5s ease-in-out;
        }
        @keyframes pulse {
            0% {
                transform: translate(-50%, -50%) scale(1);
            }
            50% {
                transform: translate(-50%, -50%) scale(1.1);
            }
            100% {
                transform: translate(-50%, -50%) scale(1);
            }
        }
    </style>
</head>
<body>
    <div class="timer-container">
```

```
    <div class="rotating-circle"></div>
    <div id="timer">3초</div>
</div>
<script>
    const timerElement = document.getElementById("timer");
    const rotatingCircle = document.querySelector(".rotating-circle");
    let seconds = 3;
    function updateTimer() {
        if (seconds > 0) {
            timerElement.textContent = `${seconds}초`;
            timerElement.classList.add("pulse");
            setTimeout(() => {
                timerElement.classList.remove("pulse");
            }, 500);
            seconds--;
            setTimeout(updateTimer, 1000);
        } else {
            timerElement.textContent = "운동을 시작해주세요!";
            timerElement.style.fontSize = "3.5rem";
            rotatingCircle.style.animation = "fadeOutSlow 1.5s forwards";
            setTimeout(() => {
                window.location.href = "/squat/";
            }, 2000);
        }
    }
```

```
        updateTimer();
      </script>
    </body>
</html>
```

2.2.4 스쿼트 화면 웹페이지 (squat.html)

- 스쿼트를 실시간으로 스트리밍하는 페이지이다.

```
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1, shrink-to-fit=no"
    />
    <link
      rel="stylesheet"
      href="https://stackpath.bootstrapcdn.com/bootstrap/4.1.3/css/bootstrap.min.css"
      integrity="sha384-
MCw98/SFnGE8fJT3GXwEOngsV7Zt27NXFoaoApmYm81iuXoPkFOJwJ8ERdknLPMO"
      crossorigin="anonymous"
    />
    <style>
      body {
        background-color: #000; /* 검정색 배경 */
```

```
        color: white; /* 텍스트 색상 흰색 */
    }
    h3 {
        text-align: center;
        margin-top: 20px;
    }
    .stream-container {
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
    }
    img {
        border: 2px solid #fff;
        border-radius: 10px;
        display: block; /* block으로 변경하여 margin 사용 가능 */
        margin-left: auto; /* 왼쪽 여백 자동 */
        margin-right: auto; /* 오른쪽 여백 자동 */
        width: 50%; /* 가로 크기를 50%로 설정 */
        height: auto; /* 세로 비율 자동 조절 */
    }
  </style>
  <title>Streaming</title>
</head>
<body>
```

```
    <!-- Socket.IO 클라이언트 라이브러리 -->
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.0.1/socket.io.min.js"></script>
    <script>
      // 서버에 연결
      var socket = io.connect('http://192.168.137.21:5000');

      // 서버에서 'redirect' 이벤트를 수신했을 때 리디렉션 처리
      socket.on('redirect', function(data) {
        if (data.redirect) {
          window.location.href = '/bye_squat'; // '/bye_squat' 페이지로 리디렉션
        }
      });
    </script>

    <div class="container-fluid stream-container">
      <div class="col-lg-10">
        <h3>실시간 스쿼트 스트리밍</h3>
        <img src="{{ url_for('stream') }}" />
      </div>
    </div>
  </body>
</html>
(myenv) pi@2171456:~/project/templates $ ^C
(myenv) pi@2171456:~/project/templates $ cat squat.html
```

```
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1, shrink-to-fit=no"
    />
    <link
      rel="stylesheet"
      href="https://stackpath.bootstrapcdn.com/bootstrap/4.1.3/css/bootstrap.min.css"
      integrity="sha384-
MCw98/SFnGE8fJT3GXwEOngsV7Zt27NXFoaoApmYm81iuXoPkFOJwJ8ERdknLPMO"
      crossorigin="anonymous"
    />
    <style>
      body {
        background-color: #000; /* 검정색 배경 */
        color: white; /* 텍스트 색상 흰색 */
      }
      h3 {
        text-align: center;
        margin-top: 20px;
      }
      .stream-container {
        display: flex;
```

```
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        img {
            border: 2px solid #fff;
            border-radius: 10px;
            display: block; /* block으로 변경하여 margin 사용 가능 */
            margin-left: auto; /* 왼쪽 여백 자동 */
            margin-right: auto; /* 오른쪽 여백 자동 */
            width: 50%; /* 가로 크기를 50%로 설정 */
            height: auto; /* 세로 비율 자동 조절 */
        }
    </style>
    <title>Streaming</title>
</head>
<body>
    <!-- Socket.IO 클라이언트 라이브러리 -->
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.0.1/socket.io.min.js"></script>
    <script>
        // 서버에 연결
        var socket = io.connect('http://192.168.137.21:5000');

        // 서버에서 'redirect' 이벤트를 수신했을 때 리디렉션 처리
```

```
        socket.on('redirect', function(data) {
            if (data.redirect) {
                window.location.href = '/bye_squat'; // '/bye_squat' 페이지로 리디렉션
            }
        });
    </script>

    <div class="container-fluid stream-container">
        <div class="col-lg-10">
            <h3>실시간 스쿼트 스트리밍</h3>
            <img src="{{ url_for('stream') }}" />
        </div>
    </div>
  </body>
</html>
```

2.2.5 스쿼트 분석 텍스트 표시 웹페이지 (bye_squat.html)

- 스쿼트 분석 결과 웹페이지에 리디렉션하기 전 "스쿼트 결과를 분석해드릴게요!"라는 텍스트를 화면에 표시한다.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
<title>활기찬 3초 타이머</title>
<style>
  body {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    background-color: #1a1a2e;
    font-family: Arial, sans-serif;
    overflow: hidden;
  }
  .timer-container {
    position: relative;
    width: 350px;
    height: 350px;
  }
  .rotating-circle {
    position: absolute;
    width: 100%;
    height: 100%;
    border: 4px solid #16213e;
    border-radius: 50%;
  }
  @keyframes fadeOutSlow {
```

```
            0% {
                opacity: 1;
            }
            100% {
                opacity: 0;
            }
        }
        #timer {
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            font-size: 5rem;
            font-weight: bold;
            color: #4ade80;
            text-shadow: 0 0 10px rgba(74, 222, 128, 0.5);
            text-align: center;
            white-space: nowrap;
        }
    </style>
</head>
<body>
    <div class="timer-container">
        <div id="timer">스쿼트 결과를 분석해드릴게요!</div>
    </div>
```

```
    <script>
      const timerElement = document.getElementById("timer");
      const rotatingCircle = document.querySelector(".rotating-circle");

      setTimeout(() => {
        timerElement.style.opacity = "0";
        setTimeout(() => {
          window.location.href = "/feedback_squat/";
        }, 1000);
      }, 3000);
    </script>
  </body>
</html>
```

2.2.6 스쿼트 분석 결과 웹페이지 (feedback_squat.html)

- 스쿼트 피드백 페이지를 구성하는 코드로, 사용자가 스쿼트 데이터를 확인하고, 목표를 설정하는 기능을 제공한다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HELLNADO - 운동 피드백</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
```

```
    <style>
        :root {
            --primary-color: #0071e3;
            --background-color: #000000;
            --text-color: #f5f5f7;
            --section-bg-color: #1d1d1f;
            --border-color: #424245;
        }
        body, html {
            margin: 0;
            padding: 0;
            font-family: "SF Pro Text", "SF Pro Icons", "Helvetica Neue", "Helvetica", "Arial", sans-serif;
            background-color: var(--background-color);
            color: var(--text-color);
            line-height: 1.47059;
            font-weight: 400;
            letter-spacing: -0.022em;
        }
        .container {
            width: 100%;
            max-width: 1440px;
            margin: 0 auto;
            padding: 0 30px;
            box-sizing: border-box;
```

```
}
.nav-wrapper {
    background-color: rgba(29, 29, 31, 0.72);
    backdrop-filter: saturate(180%) blur(20px);
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    z-index: 9999;
}
nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
    height: 44px;
    padding: 0 30px;
}
nav a {
    color: var(--text-color);
    text-decoration: none;
    font-size: 12px;
    opacity: 0.8;
    transition: opacity 0.3s ease;
    display: flex;
    align-items: center;
```

```
}
nav a:hover {
	opacity: 1;
}
nav a i {
	margin-right: 5px;
}
main {
	padding-top: 88px;
}
h1, h2, h3 {
	font-weight: 600;
}
h1 {
	font-size: 56px;
	line-height: 1.07143;
	letter-spacing: -0.005em;
	text-align: left;
	margin-bottom: 40px;
}
h2 {
	font-size: 28px;
	line-height: 1.14286;
	letter-spacing: 0.007em;
	margin-bottom: 20px;
```

```
}
.feedback-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 30px;
}
.feedback-section {
    background-color: var(--section-bg-color);
    border-radius: 18px;
    padding: 30px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    transition: all 0.3s ease;
    animation: fadeInUp 0.6s ease-out forwards;
    opacity: 0;
}
.feedback-section:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 12px rgba(0, 0, 0, 0.2);
}
.feedback-section.wide {
    grid-column: span 2;
}
#videoPlayer {
    width: 100%;
    border-radius: 12px;
```

```
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
.progress-bar {
        background-color: var(--border-color);
        border-radius: 100px;
        height: 6px;
        width: 100%;
        margin-top: 10px;
        overflow: hidden;
}
.progress {
        background-color: var(--primary-color);
        height: 100%;
        border-radius: 100px;
        transition: width 0.5s ease-in-out;
}
.goal-input {
        display: flex;
        align-items: center;
        margin-top: 20px;
}
.goal-input input {
        flex-grow: 1;
        padding: 12px 16px;
        border-radius: 8px;
```

```
        border: 1px solid var(--border-color);
        background-color: var(--section-bg-color);
        color: var(--text-color);
        font-size: 17px;
        line-height: 1.23536;
        font-weight: 400;
        letter-spacing: -0.022em;
}
.goal-input button {
        background-color: var(--primary-color);
        color: white;
        border: none;
        padding: 12px 20px;
        border-radius: 8px;
        cursor: pointer;
        font-size: 17px;
        line-height: 1.17648;
        font-weight: 400;
        letter-spacing: -0.022em;
        margin-left: 10px;
        transition: background-color 0.3s ease;
}
.goal-input button:hover {
        background-color: #0077ed;
}
```

```
@media (max-width: 1200px) {
    .feedback-grid {
        grid-template-columns: repeat(2, 1fr);
    }
    .feedback-section.wide {
        grid-column: span 1;
    }
}
@media (max-width: 768px) {
    .feedback-grid {
        grid-template-columns: 1fr;
    }
    h1 {
        font-size: 40px;
    }
}
@keyframes fadeInUp {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
```

```
            }
            .feedback-section:nth-child(1) {
                animation-delay: 0.1s;
            }
            .feedback-section:nth-child(2) {
                animation-delay: 0.2s;
            }
            .feedback-section:nth-child(3) {
                animation-delay: 0.3s;
            }
            .feedback-section:nth-child(4) {
                animation-delay: 0.4s;
            }
            .feedback-section:nth-child(5) {
                animation-delay: 0.5s;
            }
        </style>
</head>
<body>
        <div class="nav-wrapper">
            <nav>
                <a href="#" onclick="window.open('/')"><i class="fas fa-dumbbell"></i>
HELLNADO</a>
                <a href="#"><i class="fas fa-user"></i>  프로필</a>
            </nav>
```

```
    </div>
    <main>
        <div class="container">
            <h1>운동 피드백</h1>
            <div class="feedback-grid">
                <div class="feedback-section">
                    <h2>운동 영상 다운로드</h2>
                    <br><br>
                    <a href="{{ url_for('serve_video', filename=video_filename) }}"
download>
                        <button>운동 영상 다운로드</button>
                    </a>
                </div>
                <div class="feedback-section">
                    <h2>주변 환경</h2>
                    <p>온도 : <span id="temperature">0</span>℃</p>
                    <p>습도 : <span id="humidity">0</span>%</p>
                    <p>촬영 거리 : <span id="distance">0</span>cm</p>
                </div>
                <div class="feedback-section">
                    <h2>운동 요약</h2>
                    <p>총 운동 시간: <span id="totalTime">0</span>초</p>
                    <p>완료한 세트: <span id="completedSets">0</span></p>
                    <p>총 반복 횟수: <span id="totalReps">0</span></p>
                </div>
```

```
                    <div class="feedback-section">
                        <h2>성과</h2>
                        <p>목표 대비 달성률:</p>
                        <div class="progress-bar">
                            <div id="achievementProgress" class="progress" style="width:
0%"></div>
                        </div>
                        <p id="achievementText"></p>
                    </div>
                    <div class="feedback-section wide">
                        <h2>목표 설정</h2>
                        <p>목표 반복 횟수를 설정하세요</p>
                        <form action="/set_squat_goal" method="post">
                        <div class="goal-input">
                            <input type="number" id="nextGoal" name="nextGoal"
placeholder="목표 반복 횟수">
                            <button onclick="sentNextGoal()">설정</button>
                        </div>
                    </div>
                </div>
            </div>
        </main>
        <script>
            const socket = io();

            socket.on('redirect_to_bye', function() {
```

```
            setTimeout(() => {
                window.location.href = "/bye_squat";
            }, 5000); // 5초 후 리디렉션
        });

        function submitGoal() {
            const goalReps = document.getElementById('goalReps').value;
            socket.emit('set_goal', { goalReps: goalReps });
        }

        socket.on('feedback_data', function(data) {
            document.getElementById('totalTime').textContent = data.totalTime;
            document.getElementById('completedSets').textContent =
data.completedSets;
            document.getElementById('totalReps').textContent = data.totalReps;

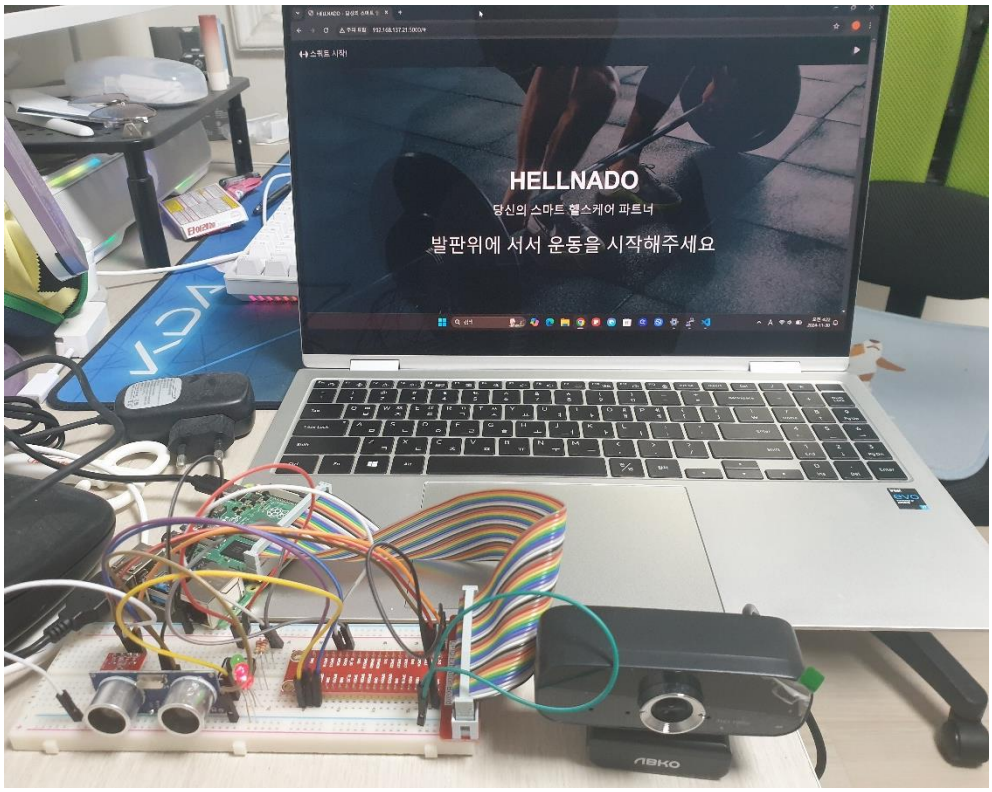
            const achievement = (data.totalReps / data.goalReps) * 100;
            document.getElementById('achievementProgress').style.width = achievement
+ "%";
            document.getElementById('achievementText').textContent = `목표 달성률:
${achievement.toFixed(2)}%`;
        });
    </script>
</body>
</html>
// 페이지 로드 시 SocketIO 연결>= 5
```

```
window.onload = function() {
    socket.emit('connect');
};
```

3. 실행 과정 및 결과

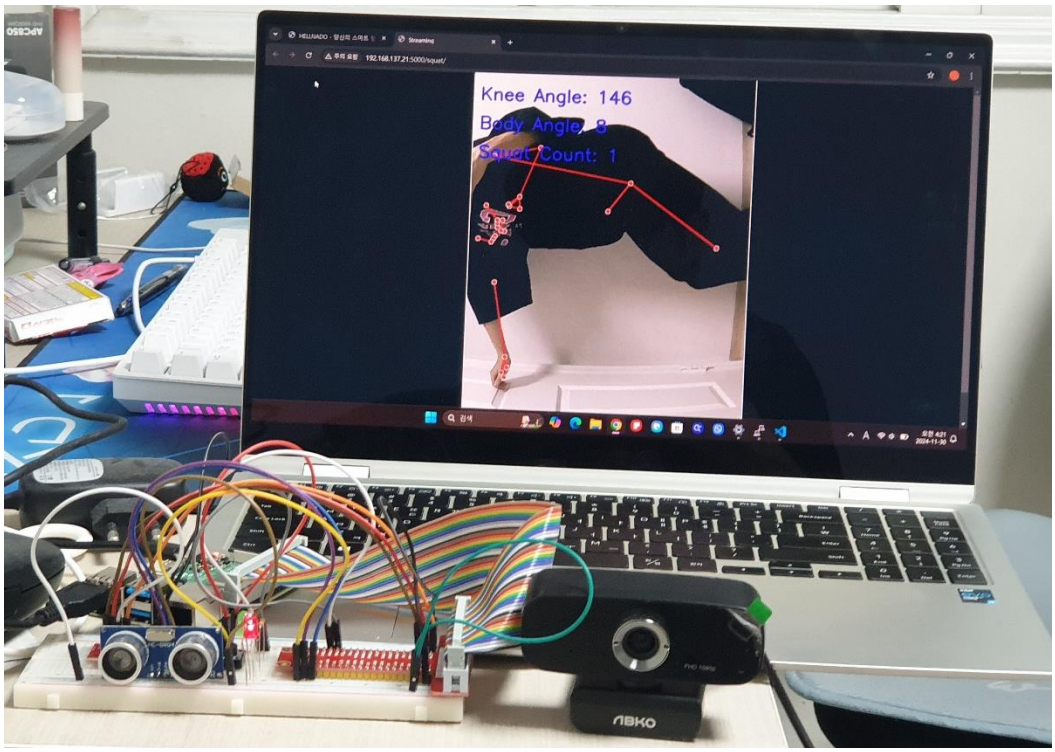
3.1 스쿼트 준비

스쿼트를 시작하기 전 카메라와 초음파 센서 앞에 선다.



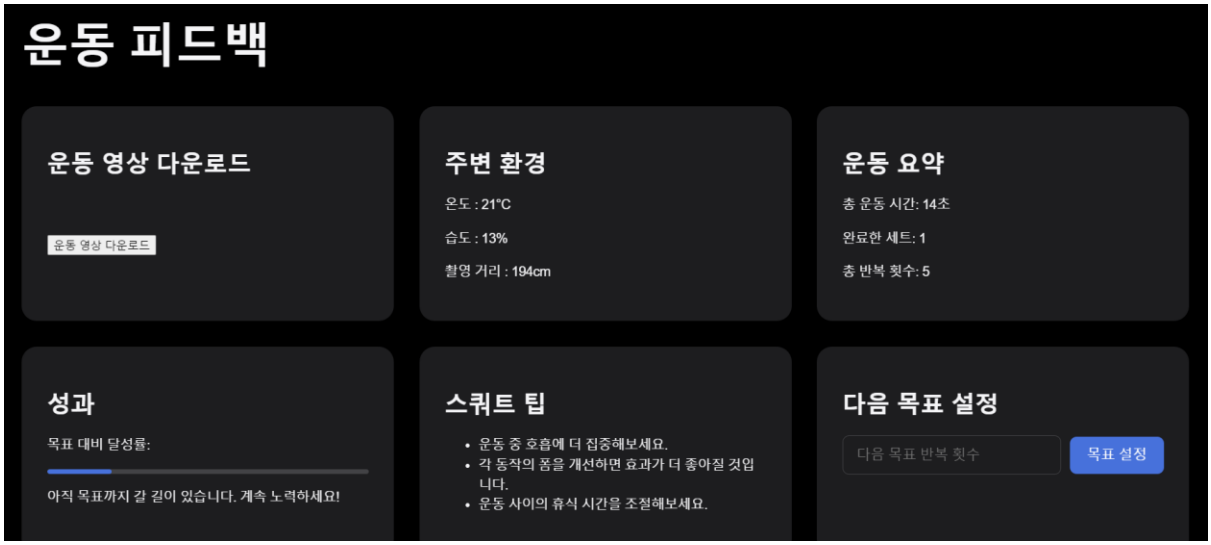
3.2 스쿼트 시작

스쿼트를 할 때 비디오 스트리밍 화면에 빨간 줄로 자세를 그리며, LED는 빨강색 LED(GPIO5)가 점등된다. 만약 올바른 자세로 스쿼트를 할 경우 선이 초록색으로 바뀌며 LED는 초록색 LED(GPIO6)가 1초 점등되었다가 꺼진다. 스쿼트를 5회 완료했을 경우 비디오 스트리밍을 멈추고 스쿼트 분석 화면으로 넘어간다.



3.3 스쿼트가 끝이 난 웹 브라우저의 모습

스쿼트 데이터를 확인할 수 있는 정보를 제공하며, 구성은 다음 사진과 같다.



4. 결론

모바일&스마트 시스템 과목을 통해 리눅스 운영체제 사용 방법에 대해 배우고 라즈베리파이의 사용 방법과 기본 원리를 익혔다.

미니프로젝트를 통해 라즈베리파이와 파이썬, HTML 파일 연동 방법을 알게 되었고, HTML 페이지에 카메라 스트리밍 출력과 스쿼트 자세 랜드마크 출력, 텍스트 출력에 어려움이 있었으며, static과 templates 라이브러리를 생성하여 이를 해결하였다.

그리고 IoT 시스템에 대한 공부의 필요성과 웨어러블 기기(스마트 워치, 블루투스 이어폰)와 스마트 TV, 스마트 냉장고 등의 IoT를 이용한 가전 제품을 더욱 편리하게 발전시키고 있는 우리나라의 IoT 개발자들이 대단함을 느꼈고, 훌륭한 컴퓨터 전문가로서 성장하기 위해 많은 하드웨어와 소프트웨어를 다뤄보며 지식을 습득해야 함을 알게 되었다.